

Documentation

Chapter 1 – Setting Up Your ROS 2 Workspace and Writing Your First Nodes

Goal: By the end of this chapter you will have a functional ROS 2 workspace containing two Python nodes—a **publisher** and a **subscriber**—and you will understand **every single line** of code that makes them tick.

1 Prerequisites

- Ubuntu 22.04 or later (ROS 2 Humble recommended)
- A terminal window and basic command-line confidence
- ROS 2 already installed and sourced (e.g. `source /opt/ros/humble/setup.bash`)

If you installed a different ROS 2 distribution, replace `humble` in the examples with the correct name.

2 Creating a Workspace

A *workspace* is simply a directory that mirrors your project. It contains your source code (`src/`), build products (`build/`) and installed artifacts (`install/`). We will call our workspace `` because later chapters will extend it into a complete mobile-robot stack.

```
# 1. Move to a convenient parent directory (adjust to taste)
$ cd ~/github/ROS2sim

# 2. Create the workspace and its source folder
$ mkdir -p bumperbot_ws/src

# 3. Step into the workspace root so colcon can find it
$ cd bumperbot_ws
```

2.1 First build (empty workspace)

`colcon build` discovers every package beneath the current directory's `src/` tree, builds them, and stages the output into `install/`.

```
$ colcon build    # Nothing to compile yet, but this prepares directory
structure
```

You should now see three folders:

```
bumperbot_ws/
├─ build/      # CMake & Python byte-code output
├─ install/    # Ready-to-run executables, libraries, resources
└─ src/        # Your source code lives here
```

3 Creating Example Packages

ROS 2 supports several *build types*. We will create **one Python package** and (optionally) **one C++ package** so you can compare idioms later.

```
# Inside bumperbot_ws/src
$ cd src/

# 3.1 Python package
$ ros2 pkg create --build-type ament_python bumperbot_py_examples

# 3.2 C++ package (optional for this chapter)
$ ros2 pkg create --build-type ament_cmake bumperbot_cpp_examples

# Return to workspace root and rebuild
$ cd ..
$ colcon build
```

After the build finishes, **always** source the local overlay so that your shell can discover the newly built packages:

```
# Option A: source via absolute path
$ source install/setup.bash

# Option B: if you are still inside install/
```

```
$ cd install/  
$ . setup.bash
```

Confirm everything worked:

```
$ ros2 pkg list | grep bumperbot  
bumperbot_cpp_examples  
bumperbot_py_examples
```

4 Writing `simple_publisher.py`

Create the

file `bumperbot_ws/src/bumperbot_py_examples/bumperbot_py_examples/simple_publisher.py`. The full code is reproduced below, followed by an exhaustive line-by-line explanation.

```
#!/usr/bin/env python3  
"""A minimal ROS 2 publisher that writes a string to /chatter once per  
second."""  
  
# 1 Import the rclpy client library (core ROS 2 Python API)  
import rclpy  
  
# 2 Import the base Node class we will inherit from  
from rclpy.node import Node  
  
# 3 Import the message type we intend to send  
from std_msgs.msg import String  
  
class SimplePublisher(Node):  
    """Publish a string message on /chatter at 1 Hz."""  
  
    def __init__(self) -> None:  
        # 4 Call parent constructor with the node name  
        super().__init__("simple_publisher")  
  
        # 5 Create a publisher handle  
        # • String: message type  
        # • "chatter": topic name  
        # • 10: size of the outgoing message queue  
        self.publisher = self.create_publisher(String, "chatter", 10)
```

```

# 6 State variables
self.counter = 0          # message number
self.timer_period = 1.0   # seconds; 1 Hz

# 7 Log a banner so the user knows the node is alive
self.get_logger().info(f"Publishing at {1/self.timer_period:.0f}
Hz")

# 8 Register a periodic callback
#   When `self.timer_period` elapses, `timer_callback` is invoked
self.timer = self.create_timer(self.timer_period,
self.timer_callback)

# 9 The callback executed every self.timer_period seconds
def timer_callback(self) -> None:
    msg = String()
    msg.data = f"Hello World {self.counter}"

# 10 Transmit the message
self.publisher.publish(msg)

# 11 Bookkeeping for the next pass
self.counter += 1

def main() -> None:
    """ROS 2 boilerplate: initialize, spin, and shut down."""

# 12 Initialize the rclpy system
rclpy.init()

# 13 Instantiate our node
node = SimplePublisher()

# 14 Enter the event loop—this blocks until Ctrl-C
rclpy.spin(node)

# 15 Clean up explicitly (optional but good style)
node.destroy_node()
rclpy.shutdown()

# 16 Allow the file to be executed directly.
if __name__ == "__main__":
    main()

```

4.1 Why Each Line Exists

Line(s)	Purpose
1 – 3	Import ROS 2 core API and the message definition we will publish.
4	Create a uniquely named node so ROS 2 can manage it.
5	Register as a publisher on <code>/chatter</code> ; the queue length of 10 buffers bursts if the network hiccups.
6	Keep state between callbacks: <code>counter</code> increments every message; <code>timer_period</code> sets publish rate.
7	Human-readable console output.
8	Arrange for <code>timer_callback()</code> to run every second.
9 – 11	Construct and publish a message, then increment the counter.
12 – 15	Required boilerplate for every rclpy application.
16	Standard Python <i>entry-point</i> check so the file doubles as a script.

5 Registering the Executable

Open `bumperbot_ws/src/bumperbot_py_examples/setup.py` and add the `simple_publisher` entry-point so that `ros2 run` can find it:

```
entry_points={
    "console_scripts": [
        "simple_publisher = bumperbot_py_examples.simple_publisher:main",
    ],
},
```

Tip – Every time you change `setup.py` or any Python source file inside a package, you must **re-build** and **re-source** the workspace.

5.1 Declaring Runtime Dependencies

Edit `bumperbot_ws/src/bumperbot_py_examples/package.xml` and add two tags **after** the `<license>` element:

```
<exec_depend>rclpy</exec_depend>
<exec_depend>std_msgs</exec_depend>
```

These inform the ROS packaging system that your node will not run unless the `rclpy` library and the `std_msgs` interface definitions are present.

Re-build and re-source:

```
$ cd ~/github/ROS2sim/bumperbot_ws
$ colcon build --symlink-install # --symlink-install speeds up Python
  edits
$ source install/setup.bash
```

6 Running the Publisher

1. Terminal 1 – Run the node

```
$ ros2 run bumperbot_py_examples simple_publisher
[INFO] [simple_publisher]: Publishing at 1 Hz
[INFO] [simple_publisher]: Sending: "Hello World 0"
...
```

2. Terminal 2 – Inspect topics

```
$ ros2 topic list
/chatter
/parameter_events
/rosout

$ ros2 topic echo /chatter
data: Hello World 0
data: Hello World 1
...
```

Congratulations! You have written and launched your first ROS 2 publisher.

7 Writing `simple_subscriber.py`

Create `bumperbot_ws/src/bumperbot_py_examples/bumperbot_py_examples/simple_subscriber.py`:

```
#!/usr/bin/env python3
"""A minimal ROS 2 subscriber that prints every String it receives on
/chatter."""

import rclpy
from rclpy.node import Node
from std_msgs.msg import String

class SimpleSubscriber(Node):
    """Log every incoming String on /chatter."""

    def __init__(self) -> None:
        super().__init__("simple_subscriber")

        # 1 Create the subscription
        self.subscriber = self.create_subscription(
            String,                    # Message type
            "chatter",                # Topic name
            self.message_callback,    # Callback executed on arrival
            10                        # Queue size
        )

        # 2 Prevent unused-variable warning (only needed in C++)
        self.subscriber # noqa: F841

        # 3 Callback function
        def message_callback(self, msg: String) -> None:
            self.get_logger().info(f"I heard: {msg.data}")

def main() -> None:
    rclpy.init()
    node = SimpleSubscriber()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()

if __name__ == "__main__":
    main()
```

7.1 Add to `setup.py`

Extend the `console_scripts` list so it now reads:

```
entry_points={
    "console_scripts": [
        "simple_publisher = bumperbot_py_examples.simple_publisher:main",
        "simple_subscriber = bumperbot_py_examples.simple_subscriber:main",
    ],
},
```

Re-build and re-source **again**:

```
$ colcon build --symlink-install
$ source install/setup.bash
```

8 Testing the Full Pipeline

1. Terminal 1 – Publisher

```
$ ros2 run bumperbot_py_examples simple_publisher
```

2. Terminal 2 – Subscriber

```
$ ros2 run bumperbot_py_examples simple_subscriber
[INFO] [simple_subscriber]: I heard: Hello World 0
[INFO] [simple_subscriber]: I heard: Hello World 1
...
```

3. Terminal 3 – Inject a manual message (optional)

```
$ ros2 topic pub /chatter std_msgs/msg/String "data: 'Manual hello'"
```

Watch Terminal 2 print the injected string.

9 What's Next?

You now own a ROS 2 workspace, understand how to add packages, and can wire up basic communication between nodes. The next chapter will delve into **parameters and QoS policies** so your nodes become configurable and reliable.

— End of Chapter 1

Chapter 2 – Building Your Robot Model with URDF

Goal: Describe your robot geometrically so that every ROS 2 tool (RViz, Gazebo, navigation stacks, etc.) can reason about its shape and kinematics.

1 What Is URDF?

Unified Robot Description Format (URDF) is an XML dialect used across ROS to model a robot's physical structure. In its simplest form a URDF file answers three questions:

Concept	URDF Element	What it encodes
<i>Rigid body</i>	<code><link></code>	Mass, inertia, visuals, and collision geometry of a single, non-deformable part.
<i>Connection</i>	<code><joint></code>	How two links move relative to one another (fixed, revolute, continuous, prismatic, etc.).
<i>Hierarchy</i>	Parent/child attributes	The tree structure that defines the robot's kinematic chain.

URDF files can become verbose, so ROS provides **xacro** (XML Macros) to enable variables, includes, and loops. A file ending in `.urdf.xacro` must be *expanded* into plain URDF at launch time, but otherwise follows the same rules.

2 Creating a *description* Package

A dedicated package keeps meshes, URDF, and textures neatly together.

```
# Inside your workspace root
$ cd src/
$ ros2 pkg create --build-type ament_cmake bumperbot_description
```

```
$ cd ..
$ colcon build
```

Why `ament_cmake` ? Although the package itself holds only data, we will rely on CMake's install rules to copy those assets into the *install space* where other packages can find them.

Directory layout (after we add files):

```
bumperbot_description/
├─ meshes/           # STL, DAE, COLLADA, PNG, ...
└─ urdf/
    └─ bumperbot.urdf.xacro
```

3 Authoring `bumperbot.urdf.xacro`

Create `bumperbot_ws/src/bumperbot_description/urdf/bumperbot.urdf.xacro` and paste the listing below. Each line is numbered so we can dissect it afterwards.

```
<?xml version="1.0"?>                                <!-- 1 -->

<robot xmlns:xacro="http://www.ros.org/wiki/xacro"    <!-- 2 -->
  name="bumperbot">

  <!-- A. Base frames -->
  <link name="base_footprint"/>                        <!-- 3 -->

  <link name="base_link">                               <!-- 4 -->
    <visual>                                           <!-- 5 -->
      <origin xyz="0 0 0" rpy="0 0 0"/>              <!-- 6 -->
      <geometry>                                       <!-- 7 -->
        <mesh
filename="package://bumperbot_description/meshes/base_link.STL"/><!-- 8 -->
        </geometry>
      </visual>
    </link>

    <joint name="base_joint" type="fixed">            <!-- 9 -->
      <parent link="base_footprint"/>                 <!-- 10 -->
      <child link="base_link"/>                       <!-- 11 -->
      <origin xyz="0 0 0.033" rpy="0 0 0"/>          <!-- 12 -->
    </joint>
```

```

<!-- B. Right wheel -->
<link name="wheel_right_link">                                <!-- 13 -->
  <visual>
    <origin xyz="0 0 0" rpy="1.57 0 0"/>                        <!-- 14 -->
    <geometry>
      <mesh
filename="package://bumperbot_description/meshes/wheel_right_link.STL"/><!--
15 -->
      </geometry>
    </visual>
  </link>

  <joint name="wheel_right_joint" type="continuous"> <!-- 16 -->
    <origin xyz="0 -0.0701101849418637 0" rpy="0 0 0"/> <!-- 17 -->
    <parent link="base_link"/>                                     <!-- 18 -->
    <child link="wheel_right_link"/>                               <!-- 19 -->
    <axis xyz="0 1 0"/>                                           <!-- 20 -->
  </joint>

  <!-- TODO: Add left wheel, caster, sensors, etc. -->

</robot>

```

3.1 Line-by-Line Explanation

Line(s)	Purpose
1	Standard XML declaration—always start URDF/xacro files with this.
2	<code><robot></code> root element; <code>xmlns:xacro</code> enables macro syntax; the <code>name</code> attribute tags every TF frame with a common prefix.
3	<code>base_footprint</code> frame lies flat on the ground; many navigation stacks expect it.
4–8	<code>base_link</code> represents the robot's rigid chassis. The nested <code><visual></code> contains display geometry only (no mass yet). Line 8 references an STL via the <i>package URI</i> syntax so that paths remain valid after installation.
9–12	A fixed joint welds <code>base_footprint</code> to <code>base_link</code> . Line 12 offsets the frame upward by 33 mm so that the footprint sits flush with the floor while the model's origin is centered on the base.
13–15	<code>wheel_right_link</code> holds the visual mesh for the right wheel. Line 14 rotates the wheel 90° (1.57 rad) about <code>x</code> so its axle aligns with <code>y</code> .
16–20	A continuous joint (unbounded rotation) connects the wheel to the chassis. Line 17 positions the wheel 70 mm to the robot's right. Line 20 declares the rotation axis—positive <code>y</code> so a positive command spins the wheel forward.

Why no `<inertial>` elements yet? For visualization alone they are optional. We will add proper masses and inertias in Chapter 3 when we simulate dynamics in Gazebo.

4 Installing Meshes and URDF Files

Open `bumperbot_ws/src/bumperbot_description/CMakeLists.txt` and add immediately after `find_package(ament_cmake REQUIRED)` :

```
install(  
  DIRECTORY meshes urdf  
  DESTINATION share/${PROJECT_NAME}  
)
```

This CMake rule copies both directories verbatim into `install/bumperbot_description/share/...` so that the `package://` URI resolves correctly at run-time.

Re-build and re-source:

```
$ colcon build  
$ source install/setup.bash
```

5 Visualising the Model in RViz

The `urdf_tutorial` package ships a convenient launch file that expands `xacro` and starts RViz configured for robot display. Install it once:

```
$ sudo apt update  
$ sudo apt install ros-humble-urdf-tutorial
```

Then launch:

```
$ ros2 launch urdf_tutorial display.launch.py \  
    model:=`pwd`/src/bumperbot_description/urdf/bumperbot.urdf.xacro
```

RViz should open with a green robot icon. Add a **RobotModel** display type if it is not present, and verify that the TF tree shows `base_footprint` → `base_link` → `wheel_right_link`.

6 Next Steps

- Add the left wheel and any caster wheels.
- Insert `<collision>` and `<inertial>` blocks for realistic physics.
- Create *xacro properties* for wheel radius, track width, and other parameters so you can tweak a single value and regenerate the whole model.

— End of Chapter 2

Chapter 3 – Working with ROS 2 Parameters

Goal: Learn how to declare, read, update, and validate parameters so your nodes become configurable at run-time.

1 What Are Parameters?

A **parameter** is a named, strongly-typed value stored inside a node's private key-value map. Parameters let you:

- **Configure** behaviour at launch (`ros2 run ... -p name:=value`)
- **Inspect** and **tune** a running node (`ros2 param get / set`)
- Persist settings in YAML files that load automatically

Supported types are **Boolean**, **Integer**, **Double**, **String**, **ByteArray**, **BoolArray**, **IntegerArray**, and **DoubleArray**.

Every node starts with an *empty* parameter set. You must explicitly `declare_parameter()` each key you intend to use—otherwise attempts to read or change it will fail.

2 Writing `simple_parameter.py`

Create `bumperbot_ws/src/bumperbot_py_examples/bumperbot_py_examples/simple_parameter.py`.

```
#!/usr/bin/env python3
"""Demonstrate declaring, reading, and dynamically updating parameters."""
```

```

# 1 Core ROS 2 Python API
import rclpy
from rclpy.node import Node

# 2 Interface types for the callback result and type checking
from rcl_interfaces.msg import SetParametersResult
from rclpy.parameter import Parameter

class SimpleParameter(Node):
    """Expose two parameters and print a message whenever they change."""

    def __init__(self) -> None:
        # 3 Initialise the Node with a unique name
        super().__init__("simple_parameter")

        # 4 Declare parameters with default values
        self.declare_parameter("simple_int_param", 28)
        self.declare_parameter("simple_string_param", "voss")

        # 5 Register a callback that fires *before* any parameter actually
        changes
        self.add_on_set_parameters_callback(self.param_change_callback)

        # 6 Validation callback. If we return success=False the change is
        rejected.
        def param_change_callback(self, params: list[Parameter]) ->
        SetParametersResult:
            result = SetParametersResult(successful=True)

            for param in params:
                # 7 Integer guard
                if param.name == "simple_int_param":
                    if param.type_ == Parameter.Type.INTEGER:
                        self.get_logger().info(f"simple_int_param changed to
{param.value}")
                    else:
                        result.successful = False
                        result.reason = "simple_int_param must be an
integer"

                # 8 String guard
                elif param.name == "simple_string_param":
                    if param.type_ == Parameter.Type.STRING:
                        self.get_logger().info(f"simple_string_param changed to

```

```

{param.value}")
    else:
        result.successful = False
        result.reason      = "simple_string_param must be a
string"

    # 9 Unknown parameter – reject
    else:
        result.successful = False
        result.reason      = f"Invalid parameter: {param.name}"

    return result

def main() -> None:
    # 10 Boilerplate lifecycle
    rclpy.init()
    node = SimpleParameter()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()

if __name__ == "__main__":
    main()

```

2.1 Line-by-Line Synopsis

Line(s)	Purpose
1–2	Import ROS 2 client library and parameter message types.
3	Initialise the node named <code>simple_parameter</code> .
4	Declare parameters with defaults—these now exist in the node's parameter server.
5	Register a validation callback executed <i>atomically</i> before parameters apply.
6–9	Inspect each incoming change; accept or reject based on name and type.
10	Standard ROS 2 start-spin-shutdown sequence.

3 Package Integration

3.1 package.xml

Add the runtime dependency (note the plural **rcl_interfaces**):

```
<exec_depend>rcl_interfaces</exec_depend>
```

3.2 setup.py

Extend the `console_scripts` list:

```
entry_points={
    "console_scripts": [
        "simple_publisher = bumperbot_py_examples.simple_publisher:main",
        "simple_subscriber = bumperbot_py_examples.simple_subscriber:main",
        "simple_parameter = bumperbot_py_examples.simple_parameter:main",
    ],
},
```

Re-build and re-source:

```
$ colcon build --symlink-install
$ source install/setup.bash
```

4 Running and Interacting

4.1 Launch with defaults

```
$ ros2 run bumperbot_py_examples simple_parameter
```

4.2 Query parameter values

```
$ ros2 param get /simple_parameter simple_string_param
# prints: "voss"
```

4.3 Override at startup

```
$ ros2 run bumperbot_py_examples simple_parameter \
    --ros-args -p simple_int_param:=30
```



```
$ ros2 param get /simple_parameter simple_int_param # prints 30
```

4.4 Change while running

```
$ ros2 param set /simple_parameter simple_int_param 34
```

Watch the node's log output confirm the change. Attempting an invalid type, e.g. `ros2 param set /simple_parameter simple_int_param "oops"`, will be **rejected** by our callback and ROS 2 will print the reason.

5 Why Use Callbacks?

- Validate ranges and types before a bad value crashes your controller.
 - Apply side-effects (e.g. resize buffers) immediately after acceptance.
 - Emit warnings or trigger events when certain thresholds change.
-

6 Next Steps

In Chapter 4 we will load parameters from YAML files and explore namespacing so multiple robot instances can coexist without collisions.

— End of Chapter 3

Chapter 4 – Visualising a Live URDF in RViz with *robot_state_publisher*

Goal: Publish your robot's TF tree from the URDF, drive joint angles interactively, and build an RViz configuration you can reuse.

1 Conceptual Flow

```
URDF (xacro) → robot_description parameter → robot_state_publisher → TF frames
```



```
joint_state_publisher / GUI →  
sensor_msgs/JointState
```

- **robot_state_publisher** – Reads the static URDF, subscribes to joint states, and continuously publishes the complete transform tree.
- **joint_state_publisher_gui** – Emits real-time `sensor_msgs/JointState` messages from a slider panel (perfect for modelling **continuous** joints such as wheels).
- **RViz** – Visualises both the TF tree and the rendered mesh geometry.

2 Launching the State Publisher

2.1 Terminal 1

```
$ ros2 run robot_state_publisher robot_state_publisher \  
  --ros-args -p robot_description:="$(xacro \  
    $(pwd)/src/bumperbot_description/urdf/bumperbot.urdf.xacro)"
```

Fragment	What it does
<code>ros2 run robot_state_publisher robot_state_publisher</code>	Execute the node.
<code>--ros-args</code>	Everything that follows is interpreted by rcl arguments parser.
<code>-p robot_description:=...</code>	Inject a <i>parameter</i> named <code>robot_description</code> .
<code>\$(xacro ...)</code>	Shell substitution: run <code>xacro</code> , convert the macro file into plain URDF, and inline the result.
<code>\$(pwd)/...</code>	Expands to an absolute path so ROS finds the file regardless of the current directory.

Tip – Wrapping the long command in a launch file is cleaner; we keep it inline here for educational clarity.

3 Publishing Joint States

3.1 Continuous wheel joints

Your URDF defines the wheels as

```
<joint name="wheel_left_joint" type="continuous">
```

A **continuous joint** has unlimited rotation but requires live angle data. Without it, every wheel frame stays frozen at 0 rad.

3.2 Terminal 2 – Slider GUI

```
$ ros2 run joint_state_publisher_gui joint_state_publisher_gui
```

Move the sliders labelled `wheel_left_joint`, `wheel_right_joint`, etc. The GUI emits a `JointState` topic that the state publisher converts into TF transforms.

4 Launching RViz

4.1 Terminal 3

```
$ ros2 run rviz2 rviz2
```

4.2 Initial RViz setup

1. **Fixed Frame** – Set to `base_footprint`, matching the root link declared in Chapter 2:

```
<link name="base_footprint"/>
```

2. **Add Displays**

- **TF** – Visualise the complete frame hierarchy.
- **RobotModel** – Choose *Description Topic* `/robot_description` (already published by `robot_state_publisher`).

Your screen should now show the chassis and a pair of wheels whose orientation updates live when you drag the GUI sliders.

5 Saving an RViz Configuration

RViz stores layouts in `~/.rviz2/`. To persist the view for teammates:

```
# Inside a fourth terminal or RViz menu: File ► Save
$ mkdir -p src/bumperbot_description/rviz
$ cp ~/.rviz2/default.rviz \
    src/bumperbot_description/rviz/display.rviz
```

Command	Explanation
<code>mkdir -p .../rviz</code>	Create a dedicated folder inside the <i>description</i> package.
<code>cp ~/.rviz2/default.rviz ...</code>	Copy the freshly saved layout so it becomes part of version control and can ship with your package.

Later you can launch RViz pre-configured with:

```
$ ros2 run rviz2 rviz2 -d $(ros2 pkg prefix  
bumperbot_description)/share/bumperbot_description/rviz/display.rviz
```

6 Troubleshooting Checklist

- **Robot invisible?** Ensure `robot_description` parameter was set (inspect with `ros2 param list /robot_state_publisher`).
- **Wheels don't rotate?** Verify `joint_state_publisher_gui` is running and publishing (`ros2 topic echo /joint_states`).
- **Mesh path errors?** Remember to `colcon build && source install/setup.bash` after adding new files so the package URI resolves.

7 Next Steps

Chapter 5 will create a launch file that starts all three nodes together and auto-loads the RViz configuration.

— End of Chapter 4

Chapter 5 – Automating Everything with a Launch File

Goal: Start `robot_state_publisher`, `joint_state_publisher_gui`, and `RViz` with a *single* command, while keeping the URDF path configurable.

1 Why Launch Files?

A ROS 2 launch file is a Python script that describes **what** nodes to run, **how** to configure them, and **when** to start them. Launch files can also **include** other launch files, set environment variables, and declare user-tunable arguments—all while remaining platform-agnostic.

Key advantages:

- One-line startup for complex systems
- Central place to document default parameters and topic remappings
- Shareable across team members without exposing absolute paths

Launch files live in a package's `launch/` directory and traditionally end in `.launch.py`.

2 Writing `display.launch.py`

Create `bumperbot_ws/src/bumperbot_description/launch/display.launch.py` and paste the code below. As usual, every line is annotated.

```
#!/usr/bin/env python3
"""Launch RViz with the robot model and GUI sliders ready to go."""

# 1 Standard libraries
import os

# 2 ament_index lets us resolve package directories at run-time
from ament_index_python.packages import get_package_share_directory

# 3 Core launch classes
from launch import LaunchDescription
from launch.actions import DeclareLaunchArgument
from launch.substitutions import Command, LaunchConfiguration

# 4 ROS-specific launch helpers
from launch_ros.actions import Node
```

```

from launch_ros.parameter_descriptions import ParameterValue

def generate_launch_description() -> LaunchDescription:
    """Compose and return a LaunchDescription object."""

    # 5 Locate the bumperbot_description package regardless of workspace
    path
    bumperbot_description_dir =
get_package_share_directory("bumperbot_description")

    # 6 User-configurable argument: path to the URDF/Xacro
    model_arg = DeclareLaunchArgument(
        name="model",
        default_value=os.path.join(
            bumperbot_description_dir, "urdf", "bumperbot.urdf.xacro"
        ),
        description="Absolute path to robot URDF or Xacro file",
    )

    # 7 Expand xacro *at launch time* and expose as a parameter value
    robot_description = ParameterValue(
        Command(["xacro ", LaunchConfiguration("model")]),
        value_type=str,
    )

    # 8 robot_state_publisher node with the generated URDF injected
    robot_state_publisher_node = Node(
        package="robot_state_publisher",
        executable="robot_state_publisher",
        parameters=[{"robot_description": robot_description}],
    )

    # 9 GUI sliders for joint positions
    joint_state_publisher_gui_node = Node(
        package="joint_state_publisher_gui",
        executable="joint_state_publisher_gui",
    )

    # 10 RViz pre-loaded with our saved layout
    rviz_node = Node(
        package="rviz2",
        executable="rviz2",
        name="rviz2",
        output="screen",
        arguments=["-d", os.path.join(bumperbot_description_dir, "rviz",

```

```

"display.rviz"]],
)

# 11 Assemble all actions into a LaunchDescription
return LaunchDescription([
    model_arg,
    joint_state_publisher_gui_node,
    robot_state_publisher_node,
    rviz_node,
])

```

2.1 Explanation Table

Line(s)	Purpose
1–4	Imports from the Python standard library and the ROS 2 launch API.
5	<code>get_package_share_directory()</code> finds the <i>installed</i> location of any package.
6	Declares a launch-time argument <code>model</code> so users can override the URDF path.
7	Uses the <code>xacro</code> CLI to convert the file into pure URDF; the result is mapped to the <code>robot_description</code> parameter.
8	Starts <code>robot_state_publisher</code> with that parameter.
9	Starts the joint-slider GUI.
10	Launches RViz and loads the layout we saved in Chapter 4.
11	Returns a list of launch actions as the <code>LaunchDescription</code> .

3 Installing Launch and RViz Assets

3.1 Amend `CMakeLists.txt`

Add `launch` and `rviz` to the install rule (after the previous chapters):

```

install(
  DIRECTORY meshes urdf launch rviz
  DESTINATION share/${PROJECT_NAME}
)

```

3.2 Extend `package.xml`

Insert these runtime dependencies **after** `<buildtool_depend>` :

```
<exec_depend>robot_state_publisher</exec_depend>
<exec_depend>joint_state_publisher_gui</exec_depend>
<exec_depend>rviz2</exec_depend>
<exec_depend>ros2launch</exec_depend>
```

Note – You do *not* need to add `xacro` ; it is part of `ros-humble-xacro` , already required by `robot_state_publisher` .

Re-build and re-source:

```
$ colcon build --symlink-install
$ source install/setup.bash
```

4 Using the Launch File

Start the entire visualization stack with a single command:

```
$ ros2 launch bumperbot_description display.launch.py
```

Pass a custom robot model if desired:

```
$ ros2 launch bumperbot_description display.launch.py \
  model:=/path/to/alternative_robot.urdf.xacro
```

You should see:

1. **Terminal output** from all three nodes.
2. **RViz** showing the robot model.
3. **Joint State GUI** for wheel sliders.

5 Next Steps

- Add a **static_transform_publisher** for sensors mounted rigidly to the chassis.
- Separate simulation-only nodes (e.g., Gazebo) into another launch file and include it conditionally.

- ## — End of Chapter 5

```
<?xml version="1.0"?>                                <!-- 1 -->
<robot name="bumperbot" xmlns:xacro="http://ros.org/wiki/xacro"> <!-- 2 -->

  <!-- 3 High-friction parameters for the drive wheels ▲▲▲ -->
  <gazebo reference="wheel_left_link">                  <!-- 3 -->
    <mu1>1e15</mu1>                                     <!-- 4 -->
    <mu2>1e15</mu2>                                     <!-- 5 -->
```

```

    <kp>1e12</kp>                                <!-- 6 -->
    <kd>10</kd>                                    <!-- 7 -->
    <minDepth>0.001</minDepth>                    <!-- 8 -->
    <maxVel>0.1</maxVel>                          <!-- 9 -->
    <fdir1>1 0 0</fdir1>                          <!-- 10 -->
</gazebo>

<gazebo reference="wheel_right_link"> ...same as above... </gazebo>

<!-- 11 Lower friction on casters so they can swivel freely -->
<gazebo reference="caster_front_link">           <!-- 11 -->
    <mu1>0.1</mu1><mu2>0.1</mu2><kp>1e6</kp><kd>100</kd>
    <minDepth>0.001</minDepth><maxVel>1.0</maxVel>
</gazebo>
<gazebo reference="caster_rear_link"> ...identical... </gazebo>
</robot>

```

Line(s)	Purpose
1	Standard XML header.
2	Root element; we only store <i>Gazebo</i> extensions here.
3–10	Wheel friction parameters. <code>mu1 / mu2</code> are Coulomb friction coefficients in the two principal directions. They're set absurdly high so the differential drive bites the ground during odometry tests. <code>kp / kd</code> form a contact "spring–damper" model to keep penetration minimal. <code>fdir1</code> forces the friction direction to align with the wheel tread.
11–	Casters get small friction so they don't steer the robot.

3 Extending the Main URDF with Inertias, Collisions, and the Gazebo Include

Open `bumperbot_description/urdf/bumperbot.urdf.xacro` and add three key changes:

1. **Include the Gazebo fragment** right after the XML header:

```

<xacro:include filename="$(find
bumperbot_description)/urdf/bumperbot_gazebo.xacro"/>

```

2. **Inertial blocks** (`<inertial>`) for every link—taken from CAD or estimated via MeshLab.
3. **Collision geometry**: we reuse the full chassis STL, but **replace the wheel meshes with simple** `<sphere>` **shapes**(radius $\approx$ wheel radius) to cut collision computation time.

An excerpt for the right wheel (annotated):

```
<link name="wheel_right_link">
  <inertial> ...mass & inertia matrix... </inertial>

  <visual>
    <origin xyz="0 0 0" rpy="1.57 0 0"/>
    <geometry><mesh filename="../../../wheel_right_link.STL"/></geometry>
  </visual>

  <collision>                                     <!-- A -->
    <origin xyz="0 -0.015 0" rpy="1.57 0 0"/>      <!-- B -->
    <geometry><sphere radius="0.033"/></geometry>    <!-- C -->
  </collision>
</link>
```

| **A** | Collision tag used only by the physics engine. | | **B** | Slight offset so the sphere hugs the tread. | | **C** | A 33 mm sphere $\approx$ wheel radius $\rightarrow$ simulation stays fast.

Repeat for the left wheel and casters.

Hint – If you later switch to *ODE* or *Bullet* inside Gazebo, primitive shapes dramatically improve contact stability over triangle meshes.

4 Creating gazebo.launch.py

Place the file in `bumperbot_description/launch/gazebo.launch.py`.

```
#!/usr/bin/env python3
"""Launch Gazebo Sim, spawn bumperbot, bridge the clock, and use
/use_sim_time."""

import os
from pathlib import Path

from ament_index_python.packages import get_package_share_directory
from launch import LaunchDescription
from launch.actions import (
    DeclareLaunchArgument, IncludeLaunchDescription, SetEnvironmentVariable,
    ExecuteProcess
)
from launch.substitutions import Command, LaunchConfiguration,
EnvironmentVariable
from launch.launch_description_sources import PythonLaunchDescriptionSource
from launch_ros.actions import Node
```

```

from launch_ros.parameter_descriptions import ParameterValue

def generate_launch_description() -> LaunchDescription:
    # Locate package share and its *parent* to expose meshes to Gazebo
    pkg_share = get_package_share_directory("bumperbot_description")
    share_root = str(Path(pkg_share).parent)

    # ----- Launch Arguments -----
    model_arg = DeclareLaunchArgument(
        name="model",
        default_value=os.path.join(pkg_share, "urdf",
"bumperbot.urdf.xacro"),
        description="Absolute path to robot URDF/Xacro file",
    )

    # ----- Environment Variables -----
    ign_resource = SetEnvironmentVariable(
        name="IGN_GAZEBO_RESOURCE_PATH",
        value=[share_root, ":",
EnvironmentVariable("IGN_GAZEBO_RESOURCE_PATH", default_value="")],
    )
    gz_resource = SetEnvironmentVariable(
        name="GZ_SIM_RESOURCE_PATH",
        value=[share_root, ":", EnvironmentVariable("GZ_SIM_RESOURCE_PATH",
default_value="")],
    )

    # Echo the result for debugging
    log_env = ExecuteProcess(
        cmd=["bash", "-lc", "echo
IGN_GAZEBO_RESOURCE_PATH=$IGN_GAZEBO_RESOURCE_PATH"],
        output="screen",
    )

    # ----- Robot Description -----
    robot_description = ParameterValue(Command(["xacro ",
LaunchConfiguration("model")]), value_type=str)

    rsp_node = Node(
        package="robot_state_publisher",
        executable="robot_state_publisher",
        parameters=[{"robot_description": robot_description, "use_sim_time":
True}],
    )

```

```

# ----- Gazebo Server & Client -----
gazebo = IncludeLaunchDescription(
    PythonLaunchDescriptionSource([
        os.path.join(get_package_share_directory("ros_gz_sim"),
"launch"), "/gz_sim.launch.py"
    ]),
    launch_arguments=[("gz_args", [" -v 4", " -r", " empty.sdf"])],
)

spawn_entity = Node(
    package="ros_gz_sim",
    executable="create",
    output="screen",
    arguments=["-topic", "robot_description", "-name", "bumperbot"],
)

clock_bridge = Node(
    package="ros_gz_bridge",
    executable="parameter_bridge",
    arguments=["/clock@rosgraph_msgs/msg/Clock[gz.msgs.Clock"],
)

return LaunchDescription([
    log_env,
    model_arg,
    ign_resource,
    gz_resource,
    rsp_node,
    gazebo,
    spawn_entity,
    clock_bridge,
])

```

4.1 Major Sections Explained

Block	What it does
<i>Environment Variables</i>	Extend IGN_GAZEBO_RESOURCE_PATH and GZ_SIM_RESOURCE_PATH. Gazebo can locate your meshes outside its default install.
robot_state_publisher	Publishes TF and /robot_description; use_sim_time=True to read the simulator clock.
ros_gz_sim ... /gz_sim.launch.py	Starts the Ignition Gazebo server (gz gui if you add -g later) with an <i>empty world</i> .
ros_gz_sim create	Spawns the robot by subscribing to /robot_description.

Block	What it does
<code>ros_gz_bridge</code> <code>parameter_bridge</code>	Relays the <code>/clock</code> topic so all ROS nodes tick in sync.

5 Package Installation Updates

5.1 CMakeLists.txt

The rule added in Chapter 5 (`launch rviz`) already installs the *launch* directory, so no change is needed.

5.2 package.xml

Add simulator dependencies:

```
<exec_depend>ros_gz_sim</exec_depend>
<exec_depend>ros_gz_bridge</exec_depend>
```

(These pull in `gz-sim`, `gz-gui`, `ign-physics` libraries, etc.)

Re-build and source:

```
$ colcon build --symlink-install
$ source install/setup.bash
```

Troubleshooting – If the linker complains about *ignition-math* versions, ensure you installed the matching `ros-humble-ros-gz-*` meta-packages for your Ubuntu release.

6 Running the Simulator

```
$ ros2 launch bumperbot_description gazebo.launch.py
```

- Gazebo GUI opens with an **empty plane**.
- The shell prints `SpawnEntity: Success` and places *bumperbot* at (0,0,0).
- The clock bridge starts; verify with:

```
$ ros2 topic echo /clock | head
```

- Inspect TF in RViz (fixed frame `base_footprint`).
- Drag wheel sliders (still running from Chapter 4) to see the robot roll.

7 Next Steps

- Add a **differential_drive_controller** via ROS 2 Control for velocity commands.
- Insert sensors (IMU, depth camera) and bridge their topics.
- Swap the ground plane for a realistic warehouse world.

— End of Chapter 6

Chapter 7 – (Optional) Running bumperbot in Gazebo Classic 11

Goal: Launch the robot in **Gazebo Classic** (gazebo-11) using a lightweight world, publish the URDF, and spawn the model—all from one launch file.

1 Why Gazebo Classic?

Although *Ignition Gazebo* (Gazebo Sim) is the future, many research stacks—and some competitions—still rely on **Gazebo Classic 11**. Classic uses a different plugin mechanism (`gazebo_ros` instead of `ros_gz_*`) and reads SDF 1.6 worlds. This chapter shows how to support both simulators side-by-side.

Key Differences

Classic 11	Gazebo Sim (Ignition)
gazebo_ros bridge plugins	ros_gz_bridge processes
SDF 1.6	SDF 1.7+
One monolithic executable (gazebo)	Separate gz server & gz sim

2 Creating a Minimal World File

Place `bumperbot_description/worlds/optimized.world`:

```
<?xml version="1.0"?>                                <!-- 1 -->
<sdf version="1.6">                                    <!-- 2 -->
```

```

<world name="empty">                                     <!-- 3 -->

  <!-- 1) Visual tweaks -->
  <scene>
    <shadows>0</shadows>                                   <!-- 4 -->
    <grid>false</grid>                                     <!-- 5 -->
    <origin_visual>false</origin_visual>                   <!-- 6 -->
  </scene>

  <!-- 2) Physics parameters (ODE) -->
  <physics name="default_physics" type="ode">             <!-- 7 -->
    <max_step_size>0.005</max_step_size>                   <!-- 8 -->
    <real_time_update_rate>200</real_time_update_rate>    <!-- 9 -->
  </physics>

  <!-- 3) Add lights / props here if needed -->
</world>
</sdf>

```

Line(s)	Purpose
1–3	Standard SDF boilerplate. The world is named <code>empty</code> .
4–6	Disable costly visuals (shadows, ground grid) for faster FPS on low-end GPUs.
7–9	Coarser physics step (5 ms) and 200 Hz update limits reduce CPU usage.

3 Install Rule Update

Edit `bumperbot_description/CMakeLists.txt` so the **worlds** folder is deployed:

```

install(
  DIRECTORY meshes urdf launch rviz worlds
  DESTINATION share/${PROJECT_NAME}
)

```

After a `colcon` build this copies all files into `install/bumperbot_description/share/...` where Gazebo will find them via `GAZEBO_MODEL_PATH` .

4 Writing gazebo_classic.launch.py

Create `bumperbot_description/launch/gazebo_classic.launch.py`:

```
#!/usr/bin/env python3
"""Launch Gazebo Classic 11, spawn bumperbot, and publish TF."""

# — Imports —
import os
from os import pathsep
from pathlib import Path

from ament_index_python.packages import (
    get_package_share_directory, get_package_prefix,
)
from launch import LaunchDescription
from launch.actions import (
    DeclareLaunchArgument, IncludeLaunchDescription, SetEnvironmentVariable,
)
from launch.launch_description_sources import PythonLaunchDescriptionSource
from launch.substitutions import (
    EnvironmentVariable, LaunchConfiguration, Command,
)
from launch_ros.actions import Node
from launch_ros.parameter_descriptions import ParameterValue

# — Launch Description Factory —

def generate_launch_description() -> LaunchDescription:
    # 1 Locate package paths
    pkg_share = get_package_share_directory("bumperbot_description")
    pkg_prefix = get_package_prefix("bumperbot_description")
    share_root = str(Path(pkg_share).parent) # ../install/share

    # 2 Expose our meshes to Gazebo via GAZEBO_MODEL_PATH
    gazebo_model_path = SetEnvironmentVariable(
        name="GAZEBO_MODEL_PATH",
        value=[share_root, pathsep, EnvironmentVariable("GAZEBO_MODEL_PATH",
default_value="")],
    )

    # 3 Launch-time argument to override the robot model
    model_arg = DeclareLaunchArgument(
        name="model",
        default_value=os.path.join(pkg_share, "urdf",
"bumperbot.urdf.xacro"),
```

```

        description="Absolute path to robot URDF/Xacro",
    )

    # 4 Include Gazebo Classic's generic launcher
    gz_ros_share = get_package_share_directory("gazebo_ros")
    world_file = os.path.join(pkg_share, "worlds", "optimized.world")
    gazebo = IncludeLaunchDescription(
        PythonLaunchDescriptionSource(os.path.join(gz_ros_share, "launch",
"gazebo.launch.py")),
        launch_arguments={
            "world": world_file,
            "paused": "false",
            "use_sim_time": "true",
            "gui": "true",
        }.items(),
    )

    # 5 Generate /robot_description
    robot_description = ParameterValue(Command(["xacro ",
LaunchConfiguration("model")]), value_type=str)
    rsp = Node(
        package="robot_state_publisher",
        executable="robot_state_publisher",
        parameters=[{"robot_description": robot_description, "use_sim_time":
True}],
    )

    # 6 Spawn the entity after Gazebo loads
    spawner = Node(
        package="gazebo_ros",
        executable="spawn_entity.py",
        arguments=["-topic", "robot_description", "-entity", "bumperbot", "-
x", "0", "-y", "0", "-z", "0.1"],
        output="screen",
    )

    # 7 Return assembled description
    return LaunchDescription([
        gazebo_model_path,
        model_arg,
        gazebo,
        rsp,
        spawner,
    ])

```

4.1 Line-by-Line Highlights

Section	Why it matters
2	Extends <code>GAZEBO_MODEL_PATH</code> so Classic finds package meshes.
4	Re-uses <code>gazebo_ros/launch/gazebo.launch.py</code> , passing our custom world.
5	Publishes TF and <code>/robot_description</code> with <code>use_sim_time=True</code> .
6	<code>spawn_entity.py</code> is Classic's factory plugin; it reads the URDF and inserts the model.

5 Building and Running

```
$ cd ~/github/ROS2sim/bumperbot_ws
$ colcon build --symlink-install
$ source /usr/share/gazebo/setup.sh      # Classic gazebo11 setup
$ source install/setup.bash
$ ros2 launch bumperbot_description gazebo_classic.launch.py
```

You should see Gazebo Classic open with the lightweight *optimized.world*, then *bumperbot* appears hovering 10 cm above the ground before settling on the plane. The ROS clock switches to simulation time, confirmed via:

```
$ ros2 param get /robot_state_publisher use_sim_time
```

6 Next Steps

- Add a **diff-drive plugin** (`libgazebo_ros_diff_drive.so`) to command wheel joints.
- Insert LIDAR or depth cameras using `<gazebo>` `<sensor>` blocks in the URDF.
- Benchmark FPS between Classic and Gazebo Sim to decide which fits your project.

— End of Chapter 7

Chapter 8 – Integrating `ros2_control` for Real-Time Actuation

Goal: Attach a **velocity controller** to the differential-drive wheels so you can command motion with standard ROS interfaces, whether you run Gazebo Classic or Ignition Gazebo.

1 What Is *ros2_control*?

ros2_control is a hardware-abstraction layer and controller framework that treats each actuator or sensor as a plugin. In simulation, *gazebo_ros2_control* (Classic) or *gz_ros2_control* (Ignition) loads a *system plugin* that implements the low-level read/write functions. Your URDF declares **which joints** expose **which interfaces** (e.g., *position*, *velocity*, *effort*), while a YAML file defines **controllers** (PID loops, diff-drive, trajectory, etc.).

Outcome of this chapter:

1. Extend the URDF with *ros2_control tags and conditional plugins for Humble (_Ignition) vs Iron-plus (Gazebo Sim)*.
2. Create a `bumperbot_controller` package containing a velocity controller and a launch file to spawn it.
3. Publish velocity commands and watch *bumperbot* roll.

2 Updating the Xacro Files

2.1 `bumperbot_gazebo.xacro`

We wrap the plugin section in **xacro conditionals**:

```
<gazebo>
  <!-- Humble → ign_ros2_control -->
  <xacro:if value="$(arg is_ignition)">
    <plugin filename="ign_ros2_control-system"
           name="ign_ros2_control::IgnitionROS2ControlPlugin">
      <parameters>$(find
bumperbot_controller)/config/bumperbot_controllers.yaml</parameters>
    </plugin>
  </xacro:if>

  <!-- Iron+ → gz_ros2_control -->
  <xacro:unless value="$(arg is_ignition)">
    <plugin filename="gz_ros2_control-system"
           name="gz_ros2_control::GazeboSimROS2ControlPlugin">
      <parameters>$(find
bumperbot_controller)/config/bumperbot_controllers.yaml</parameters>
    </plugin>
```

```
</xacro:unless>
</gazebo>
```

- `is_ignition` will be *true* for Humble (uses `ros_gz_sim` packages) and *false* for newer distros.
- The `<parameters>` tag points to the YAML file we will create in Section 3.

2.2 bumperbot_ros2_control.xacro

Declare the control **system** and interfaces for each wheel:

```
<ros2_control name="RobotSystem" type="system">
  <!-- Conditional hardware plugin inserted via include above -->

  <!-- Right wheel -->
  <joint name="wheel_right_joint">
    <command_interface name="velocity">
      <param name="min">-1</param><param name="max">1</param>
    </command_interface>
    <state_interface name="position"/>
    <state_interface name="velocity"/>
  </joint>

  <!-- Left wheel -->
  <joint name="wheel_left_joint"> ...same... </joint>
</ros2_control>
```

Why velocity? A differential-drive robot usually receives linear & angular velocity commands that translate to wheel angular rates.

2.3 Include Blocks in the Main URDF

At the top of `bumperbot.urdf.xacro` :

```
<xacro:arg name="is_ignition" default="true"/>
<xacro:include filename="$(find
bumperbot_description)/urdf/bumperbot_gazebo.xacro"/>
<xacro:include filename="$(find
bumperbot_description)/urdf/bumperbot_ros2_control.xacro"/>
```

No other changes are required; the wheel joints retain their existing inertial and collision definitions.

3 Creating bumperbot_controller Package

```
$ cd ~/github/ROS2sim/bumperbot_ws/src
$ ros2 pkg create --build-type ament_cmake bumperbot_controller
$ colcon build --symlink-install
```

3.1 YAML Configuration

Create `bumperbot_controller/config/bumperbot_controllers.yaml`:

```
controller_manager:
  ros__parameters:
    update_rate: 100    # Hz
    use_sim_time: true

    joint_state_broadcaster:
      type: joint_state_broadcaster/JointStateBroadcaster

    simple_velocity_controller:
      type: velocity_controllers/JointGroupVelocityController
      joints:
        - wheel_left_joint
        - wheel_right_joint
```

3.2 Package Installation Rules

Edit `bumperbot_controller/CMakeLists.txt`:

```
install(
  DIRECTORY config launch
  DESTINATION share/${PROJECT_NAME}
)
ament_package()
```

And `bumperbot_controller/package.xml` to add the launch dependency:

```
<exec_depend>ros2launch</exec_depend>
```

4 Controller Launch File

Create `bumperbot_controller/launch/controller.launch.py`:

```
#!/usr/bin/env python3
"""Spawn joint_state_broadcaster and the wheel velocity controller."""
from launch import LaunchDescription
from launch_ros.actions import Node

def generate_launch_description():
    broadcaster = Node(
        package="controller_manager",
        executable="spawner",
        arguments=["joint_state_broadcaster", "--controller-manager",
"/controller_manager"],
        output="screen",
    )
    velocity_ctrl = Node(
        package="controller_manager",
        executable="spawner",
        arguments=["simple_velocity_controller", "--controller-manager",
"/controller_manager"],
        output="screen",
    )
    return LaunchDescription([broadcaster, velocity_ctrl])
```

5 Installing Dependencies

For **Humble** (Ignition plugin):

```
$ sudo apt install ros-humble-ign-ros2-control ros-humble-ign-ros2-control-
demos
```

For newer distros the equivalent packages are `ros-iron-gz-ros2-control ...`.

If you encounter plugin load errors (Failed to load system plugin ...), ensure your system is fully updated:

```
$ sudo apt update && sudo apt full-upgrade
```

6 Build & Run

```
$ colcon build --symlink-install
$ source install/setup.bash
```

Open **three terminals** and source each:

Terminal	Command
1	<code>ros2 launch bumperbot_description gazebo.launch.py</code>
2	<code>ros2 launch bumperbot_controller controller.launch.py</code>
3	Testing commands (below)

6.1 Verify Controllers

```
$ ros2 control list_controllers
# EXPECTED
# joint_state_broadcaster [active]
# simple_velocity_controller [active]
```

6.2 Drive the Robot

```
$ ros2 topic pub /simple_velocity_controller/commands \
  std_msgs/msg/Float64MultiArray \
  "{data: [1.0, 0.0]}" # left wheel fwd, right wheel stop
```

Change the two values to `[1.0, 1.0]` to drive forward, `[1.0, -1.0]` to spin in place, etc.

6.3 Visualise the Graph

```
$ ros2 run rqt_graph rqt_graph
```

You will see the controller topics interconnected with Gazebo's joint state and command interfaces.

7 Next Steps

- Replace the simple velocity controller with **diff_drive_controller** for Twist messages on `/cmd_vel`.
- Add **PID gains** in the YAML for tighter velocity control.
- Use **controller_manager spawner** within the main Gazebo launch so two terminals become one.

— End of Chapter 8